

Chapter 1

Engineering Software

Software engineering is a relatively young engineering discipline. It originated as a sub-field within the field of computer science and evolved into a discipline in its own right as software systems became too large and too complex to be managed using ad-hoc processes and practices. Software engineering is a field in which both computing and engineering principles are applied to the development and maintenance of software systems. The tools and practices employed by software engineers enable large teams of software developers and other business and software specialists, e.g., designers, architects, test engineers to work collaboratively to design, build and maintain the large, complex software systems in place today (and which are likely to grow in size and complexity in the coming years). The field of software engineering is a rapidly evolving area of study. The tools, technologies, and practices we use to build and maintain software are continuing to evolve as we learn how to build software more efficiently and more effectively. At the same time, the nature of software itself is evolving to encompass applications far more diverse types of computing devices and to account for changing expectations of what software can and should do.

This chapter explores what it means to “engineer software” and why we need software engineering techniques and tools. It also provides a brief introduction to the essential software engineering skills and tools you will need to become a successful software engineer. At the end of this chapter, you should be able to:

1. Explain what software is and why it is not really ‘soft’.
2. Describe the high-level tasks performed by software engineers in addition to writing code.
3. Explain why software engineering is essential for building correct, efficient and robust software.
4. Describe the essential skills necessary to be a successful software engineer.

1.1 What is Software?

Software is at the heart of nearly every system in modern day life. It enables our phones to make calls (and perform many other services), our appliances to cook our meals and wash our clothes, and our cars to operate safely and efficiently. We use software from the time we wake up in the morning (e.g., an alarm-clock app on our phone) until we go to bed at night (e.g., a music app on our tablet device). We also use software to communicate with family, keep up with current events, and to socialize with friends who may live in another location. Sometimes we use software in less obvious ways. For instance, most home appliances contain software to operate the device. Software also regulates the engine, manages the brakes, and triggers the safety features in nearly every car built today. Software is also used to control traffic-light systems,

medical devices, and the lights, heating, air conditioning and door locks in many office and commercial buildings. In each of these examples, software is an enabling technology that has improved on or replaced another technology, or replaced a previously manual process. Software can also be used to improve existing technologies. For example, software in headphones is used to cancel background noise, and software in a camera can autofocus an image.

To better understand the many different types of software in use today, it is helpful to discuss categories of software. One approach to categorizing software is to consider its function. Using this schema, we can classify software into three general categories: system software, application software, and software libraries. *System software* is software that directly operates the computer hardware. For example, the operating system on your laptop is a type of system software that is actually a collection of software programs that manage the resources on your laptop (e.g., storage space) and provide common resources for other software running on the device. Examples of operating systems include Linux, Android, Mac OS, and Microsoft Windows. Users typically do not interact directly with system software. Instead, they work primarily with application software. *Application software* describes the family of programs that perform a specific task beyond the basic operation of the computer itself. Each Android or iOS app can be thought of as application software. Other applications include spreadsheets, word-processing programs, and computer games. Special kinds of application software, referred to as *plugins* or *extensions*, modify or add functionality to another program. The third category of software is referred to as *software libraries*. This type of software plays a supporting role in system and application software by providing a solution to a specific problem that is available through a well-defined *Application Programming Interface (API)*. Software libraries are built for reuse, but are generally not intended to be used stand-alone (i.e., they are intended to be used only by other software). *Software frameworks* are a special kind of software library. One key difference between a framework and a regular software library is the locus of control; in the case of a library, the software calling the library retains control of how the program executes, but in the case of a framework, the software using the framework cedes control of how the program executes to the framework.

We can also classify software according to characteristics of the hardware it is written for and the domain in which the software is used. For example, *Embedded system* software applications reside as firmware on a device dedicated to a particular use, e.g., inside a car, aircraft or medical device. Embedded system software is often developed to operate in real-time, meaning that it must meet specified time constraints to support mission-critical applications such as antilock brakes or pacemakers. These programs require careful instruction selection to ensure that deadlines are not missed or missed very infrequently. *Mobile apps*, such as those running on a phone or tablet, are another type of software. Development of these apps often must take into account the limited memory and power available on the mobile device. *Distributed or parallel software applications* are developed to solve problems that have been divided into many tasks. Each task is performed on a different processor, where the processors may or may not reside on the same computer or share the same memory space. These programs require additional instructions to coordinate the tasks.

Up to this point, we have talked about what software *does*, but we have not really defined what software *is*. Historically, the term *software* referred to the code (e.g., *source code*) used to instruct or control *hardware*. The ‘soft’ in software was intended to describe the relative ease with which the code could be changed as opposed to changing the electronic components. Hardware is viewed as more difficult to alter because reprogramming hardware entails physical modifications to the electronics, whereas reprogramming software requires only changes to text files.¹ In recent years, the term software has taken on new meaning to reflect the wide and diverse range of systems that can be modeled using computational resources. The definition of software has also evolved to account for the many *artifacts*, in addition to code, that are created during the design, development, testing, deployment, and maintenance of computing and electronic systems.

¹In the following sections we will explore how in practice, modern-day software is not as ‘soft’ as one might think. While the act of modifying one or more lines of code in an editor is trivial, the work behind the scenes to determine what line or lines of code should be changed and how to change them—not to mention the effort to ensure that these modifications both accomplish their goal and avoid other, unintended ramifications—can require significant time and effort.

For instance, some of the non-code software artifacts that may be developed as part of the engineering process include:

- *Specifications* that describe the system that is to be developed,
- *Project Planning and Tracking Documents* that help developers determine and record the scope, schedule, and resources necessary for the project, and then to track project progress,
- *Build Scripts* that are used to specify how the software is compiled, packaged, re-built, etc.,
- *Software Design Documents* that describe features of the design and supporting information such as constraints, assumptions, dependencies, and other information to help others understand the design or reasons for why a particular design was chosen,
- *Test Cases* that help assess the quality of software applications (and may or may not be written as code), and
- *User Guides* that help users understand features and changes to the software application.

In future chapters, we will look more closely at the various types of software artifacts and how each contributes to the engineering of correct, reliable and efficient software systems.

1.2 What is Engineering?

The term *engineering* is derived from the Latin term *in gignere*, meaning “to contrive or devise.” It is a creative activity that involves the application of mathematical and scientific principles to the design, development, maintenance and improvement of everything from physical structures to processes that define how something is built to software systems that control aircraft flight control systems. Engineers in all disciplines use their knowledge of math and science to find practical and cost-effective solutions to real-world problems. Prior to constructing a solution (e.g., a building, a process, or a software system) engineers will often first create a mathematical model of the problem. Potential solutions are then analyzed using the model to evaluate different design choices. Disciplined and rigorous processes are used throughout the design and building phases to help ensure the solution works as intended and that it is safe and maintainable.

A crucial aspect of engineered solutions is that they not only solve a particular problem, but that they are also designed and built with a variety of situations that may impact the integrity or functionality of the end product in mind. For instance, a structural engineer designing a bridge will prepare the bridge to withstand various loads and weather conditions. They will include tolerances to account for possible construction flaws and consider how future maintenance will affect the bridge’s properties. They will also develop a good understanding of what situations they have not prepared for, situations in which it would be unsafe to trust the bridge. Similarly, a software engineer will consider safety, security, reliability, usability, performance, and other quality dimensions when designing and building a software system.

1.3 What is Software Engineering?

Now that we have defined the terms *software* and *engineering*, we can begin to explore software engineering as a field of study and practice. As you might expect, software engineering is the disciplined application of math and scientific principles and processes to the design, development, maintenance, and improvement of software. In the early days of computing and software development, software engineering practices received much less attention. Initially, software applications were small enough to be written by an individual or a small number of programmers using ad-hoc processes and basic text editing tools. Over time however,

software became more pervasive, more complex, and required larger and larger teams to design, develop, and maintain. Eventually, the use of engineering practices and techniques became essential in order to efficiently coordinate and manage the development and maintenance of the software. Although software is not always delivered on time or within budget, even today, the software engineering technologies and tools currently available have enabled software development teams to build, deploy and maintain complex, reliable and safe software systems that would not have been possible just a few short years ago.

Most engineering disciplines are physics-based and involve the engineering of physical objects. For example, a closely related field to software engineering is the field of *computer engineering*. Like software engineers, computer engineers are concerned with the engineering of software; however, computer engineers are also interested in the hardware components. Computer engineers study many of the same topics as computer scientists and software engineers, such as data structures and algorithm design. They also study how electronic components are designed and integrated, and how software is designed, built, and optimized for specific hardware components. Computer engineers design and build custom systems ranging from the *embedded systems* found in cars, aircraft, video games, medical devices, and just about every electronic device in our homes, to super computers and clusters of computers that are the backbone of nearly every business organization.

The relatively new field of *software engineering* can be viewed as an information-based discipline, rather than a physics-based engineering discipline. Software engineering is focused on the processes and techniques for designing, building, and maintaining software. It is closely related to the field of computer science, which covers the core concepts and techniques for making a computer do something. To better understand the relationship between computer science and software engineering, consider the relationship between chemistry and chemical engineering. Chemists study the structure of chemicals, how they interact, and the theory behind chemical behaviors. Chemical engineers develop familiarity and expertise in chemistry so that they can apply the results of what chemists have discovered to solve a variety of problems that occur in the real world. Thus the relationship between the study of chemistry and the study of chemical engineering is one where chemical engineers view chemistry as a tool to solve problems, whereas chemists view chemistry as the object of study.

We can see a similar relationship between computer science and software engineering. Computer science courses focus on the theory of how computers and networks work and how they can be controlled through computer applications, as well as the fundamentals of algorithms and data structures. Computer science courses also teach us how to use computers in very sophisticated ways to build operating systems and databases, perform image processing, control robots, solve complex scheduling tasks, write algorithms that can be run in parallel or on distributed computers, and much more. Software engineers study many of these same topics to develop familiarity and expertise with computing concepts. They also study topics specifically targeting engineering aspects of how software is designed, built, and maintained in a systematic, controlled, and efficient manner, such as software testing and analysis, software architecture, and requirements engineering. In other words, software engineers study techniques and tools for applying what computer scientists have discovered to real-world problems.

A common misconception about software engineering, and software development in general, is that developers spend most of their time programming (i.e., writing code). In reality, writing code is only one of the many tasks performed by software engineers and software developers. Software engineers often spend time researching ways to solve problems and planning how they will change the code. They also read code to understand the current functionality and how it can be changed, or how it may be impacted by a change. They design testing strategies, write test cases and run the tests to determine if the changes have the intended results. They also spend time meeting with clients or team members to discuss software changes and new functionality. Software engineers also spend time tracking down *software bugs*, and reviewing changes made by other developers in order to provide feedback on the correctness and efficiency of the changes. The set of tasks performed by a software engineer varies from project to project depending on the needs of the project and may include other technical and management tasks.

Now that we have identified some of the common tasks performed by an individual software engineer, let's take a step back and look at how these tasks fit into the overall process of engineering software. Software engineering processes vary, depending on which *software development methodology* is followed by the team when planning, building, analyzing and deploying the software. Before exploring a few specific processes, let's first identify the core set of activities common to all software engineering processes:

- *Requirements specification and analysis* activities identify what the software should do.
- *System architecture and design* activities determine how the software should be partitioned into modules and how the modules interface (i.e., interact) with one another.
- *Construction* activities are primarily focused on writing or generating code.
- *Testing and analysis* activities check the software system for correctness, maintainability, performance, security, and other qualities.
- *Deployment* activities prepare the software for delivery and distribute the software to the target computer.
- *Maintenance* activities keep the software running once it is deployed.

Although there appears to be a natural ordering to the activities listed above (e.g., we cannot write software if we do not know what it is supposed to do), there are actually many ways in which these activities can be combined to engineer software. For instance, the *waterfall model* represents a sequential software development methodology where the core set of software development activities is treated as a sequence of phases. The phases may overlap, but the main idea is that the requirements are known and specified early in development and that they remain relatively unchanged. One of the main problems with software developed following this approach is that all of the requirements are rarely known upfront which, historically, has led to costly delays and rework as new requirements were discovered during latter activities. Another problem is that delaying the majority of the testing activity until late in the process increases the cost of fixing the problems that are detected during testing. The waterfall model has largely been superseded by processes that rely on multiple iterations of the activities listed above to incrementally build the system.

Evolutionary software development methodologies combine iterative and incremental approaches to software development. The basic idea behind *evolutionary software development* is to construct software by repeatedly adding small, manageable parts, performing all or most of the activities on each iteration. It also involves refining parts of the system as more information becomes available. This approach mitigates both of the problems mentioned above. First, it enables requirements and information about the system to be discovered as the software is designed, built, and tested rather than expecting the client to know all of the requirements upfront. Evolutionary software development also incorporates testing and analysis activities early in the development cycle, when it is easier and less costly to fix problems with the code. Both of these features of evolutionary development can reduce the cost of rework significantly. *Agile software development* methodologies take evolutionary development a step further by focusing on working software over comprehensive documentation and by relying on continuous feedback from the project stakeholders.

Up to this point, our discussion of software engineering has primarily focused on technical activities. However, there are also many important non-technical tasks involved with engineering software. These tasks include project planning and tracking, identifying and managing risks, communicating with project stakeholders, and documenting various parts of the project. In future lessons we will explore both types of activities in more detail.

1.4 The Need for Software Engineering

Software has a reputation for being late, over budget, and buggy (i.e., unreliable or incorrect). Sometimes, new software systems are even abandoned before they are delivered to the client. However, many software systems have been successfully deployed and maintained for many years. Well-engineered software, just as a well-engineered bridge or building, is not guaranteed to be successful. The application of rigorous and disciplined engineering practices can help offset many of the situations that keep projects from being successful. In this section we explore several aspects of software and software development that motivate the use of software engineering practices to help improve the likelihood of a delivering a robust, reliable and maintainable software system on time and within budget.

In some respects, engineering software is similar to engineering a physical structure, such as an office building. Like an office building, **software is large and complex**. Even if you already have some programming experience, it can be difficult to imagine the magnitude of the challenges that face software engineers until you study a real-world project. These challenges include developing and maintaining software systems consisting of huge numbers (hundreds of thousands or even millions) of lines of code, spread across many (hundreds or even thousands of) modules and packages. Many systems are written in multiple languages, and almost all software in place today relies on one or more libraries or frameworks. Recall too, that software is more than just code; it is a complex set of artifacts that include plans, design documents, specifications, tests, scripts, and many other types of artifacts that are created to ensure the software system operates correctly and efficiently. As you will soon learn, disciplined software engineering processes and supporting tools are essential for organizing, coordinating, and managing large complex software systems.

Although engineering software is similar to engineering a building or other physical structure, it also has some important differences. For instance, unlike engineering a physical structure, **software is expected to be malleable, or easily changed since it is constructed through an evolutionary process**. However, changing software can be risky. Source code changes in particular have the potential to affect the way the application executes or operates; a change to one line of code can impact many features in an app and can introduce bugs in unexpected parts of the program. Non-source code changes typically do not affect how the application works, but they can. For example, a correction to the user documentation or a change to a comment in the code will not affect the operation of the application, but an incorrect edit to a *build script* could reference the wrong version of a library and ultimately introduce bugs.

Updating source code requires relatively little cost and effort. The text that represents the application source code is replaced with new text (e.g., in the case of a bug fix) or new code is added (e.g., in the case where a new feature is added). This effort is similar to replacing or adding a sentence or a paragraph in a written report using a word-processing app. Modifying the source code, however, is only one step in the process of fixing a bug or implementing a new feature. It is also often the easiest and least costly step in the process. To illustrate, consider Table 1.1 which shows a subset of the software engineering activities that are performed when a user reports a bug in a software project's *issue tracker*. The first column in the table indicates the order in which the activities are performed. The second column lists who is responsible for performing the activity. The third column describes the activity performed, and the last column indicates an *artifact* that is used or affected by the activity. Notice that only one activity involves actually modifying the code to fix the bug, and that activity (#8) occurs in the middle of the process.

The process shown in Table 1.1 begins when a user reports a bug as an issue in the project's issue tracker (Activity #1). At this point, we assume a project manager assigns the issue to a software developer (Activity #2). Before the developer can fix the problem, he or she will likely begin by running the app to reproduce the bug first. This allows the developer to verify that the problem is an actual bug and to collect information about the failure (Activities #3 and #4). The developer will then use this information to locate the source of the bug (Activity #5) and to determine how and where to fix it (Activity #6). During Activity #7, the proposed bug fix is reviewed by another developer before it is implemented to help ensure the change is correct and will not adversely affect other parts of the software. After the fix is implemented, it is reviewed

#	Actor	Activity	Affected Artifacts
1	User	Reports the bug	Issue in the Issue Tracker
2	Project Manager	Assigns the issue to a software developer	Issue in the Issue Tracker
3	Developer	Reproduces the bug to observe the failure and and collect information about the failure	App
4	Developer	Determines if the bug is both an actual bug and distinct from other known bugs	Code
5	Developer	Determines the source of the problem	Code
6	Developer	Designs a solution to fix the bug	Code, Design Documentation
7	Developer	Reviews the proposed solution	Code, Design Documentation
8	Developer	Implements a solution	Code
9	Developer	Reviews the implemented solution	Code
10	Developer	Writes new test cases and/or corrects old test case(s)	Test Case(s)
11	Developer	Documents the solution and test cases	Issue in the Issue Tracker, Test Documentation
12	Developer	Tests the app	App, Test Case(s)
13	Developer	Releases the code and closes the issue	App, Issue in the Issue Tracker

Table 1.1: Software engineering tasks to fix a bug.

again (Activity #9), tested (Activities #10 and #12), and documented (Activity #11) before the the issue is closed and the code is released for use in the production system (Activity #13). If a problem is detected along the way, the change may need to be re-designed, re-implemented, or re-tested, requiring earlier steps in the process to be repeated. Although many of the activities listed in Table 1.1 are performed by a software developer, not all activities are necessarily performed by the same developer, or even by an individual developer. For instance, the developer, or developers, who review the implemented solution (Activity #9) would most likely be different from the developer who implemented the solution (Activity #8) in order to increase the value of the review and the likelihood of detecting flaws in the implementation before it is deployed.

The activities listed in Table 1.1 reflect a subset of the activities involved in fixing a bug. A similar process would be followed when a new feature is added to the software. In this case, the developer(s) determine how and where to add the feature before the changes are made to the code, and similar costs accrue. An actual software development process used in practice would include additional activities and costs, some of which are much less obvious. For instance there are many activities and costs related to project scheduling and estimation, and other less obvious costs such machine time, the cost to maintain consistency across software artifacts (e.g., a code change may the need to modify other artifacts to keep the app and the documentation consistent), and the potential costs associated with lost revenue and or future sales when buggy software is deployed.

Lost opportunities (e.g., other features that could be implemented) and future maintenance costs are other potential costs in practice. For example, when the design of the software does not lend itself to modification,

future changes become more difficult and therefore costly. Furthermore, the cost of the change may not be realized until long after the changed code is deployed. Consider a change to an online shopping application in a feature that is not often used, but eventually prevents a user from completing her purchase. In this case, there is likely lost revenue. There may also be a significant cost in fixing the bug if the developer of the code is no longer available to fix it (and even if she is, she may no longer be familiar with the code).

The sheer number of activities and costs surrounding any software change should be sufficient to dispel any notion that software is “soft”. However, we can take things even a step further and note that there are costs associated with there being so many costs! For instance, an additional cost results from the fact that it can be very difficult to estimate the amount of time for each of the activities when there are so many unknowns. We often do not know how long it will take to locate the source of a bug, we may also have trouble estimating the amount of effort to fix a bug, etc. As a result, activities such as scheduling and planning may be impacted by some unknown amount.

Now that we have identified some of the activities and costs of associated with changing code, let’s explore the different drivers of change (i.e., the motivations behind different software changes). Some changes are beyond the developer’s control. For instance, some changes are due to the evolution of requirements or to updates to how those requirements are documented and communicated (e.g., *requirements changes* and *specification changes*). In other cases, software changes are initiated by the developer. Software development, like other creative activities (e.g., writing a book), involves a certain amount of trial-and-error, both with regards to individual components and to the way those components are organized into a whole. So, regardless of the formal software development process followed or the software artifact that is being developed, software engineers often leverage what they learn by evolving and changing their solution (e.g., *design changes* and *bug fixes*).

- *Requirements Changes*: The requirements imposed on the software can change for many reasons. New features might be needed, or old features might become obsolete. Additional clients may start using the software, or prior clients may stop using it. Requirements might also change as clients’ or developers’ understanding of the problem shifts (e.g., as a result of experimenting with models prior to release or after seeing the software in action after deployment). Furthermore, not all requirements come from the customer. The environment in which the software is deployed affects its operation and can in fact be a driver for change. For example, most applications rely on services or libraries provided by the underlying operating system (OS); changes to the software may be necessary when old services are discontinued and new services are introduced. Software evolution in response to a changing needs and changing environments is often referred to as *adaptive maintenance*.
- *Specification Changes*: The problems solved by modern software systems are exceedingly complicated; it is generally not possible for one person to keep all of a system’s software requirements in mind at once, which means that these requirements must be documented as a *specification*. Sometimes a specification will change in meaning even when the requirements do not, in order to correct errors, remove inconsistencies, reduce ambiguity, etc. These changes, usually a type of *corrective maintenance*, can in turn entail corresponding modifications to the code, especially in cases where prior misunderstandings or miscommunication of the requirements led to their incorrect implementation.
- *Design Changes*: Design changes are often initiated to better bridge specifications (i.e., what is desired) with what is delivered. These changes may affect large portions of the code and involve considerable risk and initial cost, but at the same time they may have considerable payoff in terms of future maintainability and performance. Such changes are referred to as *perfective maintenance* or, in cases where they stave off potential future problems, *preventative maintenance*. Oftentimes, the goal of such maintenance is to improve the code without affecting actual functionality (i.e., to *refactor* the code).
- *Bug Fixes*: One of the most common motivations for changing software after it is deployed is to fix bugs resulting from an incorrect implementation of the requirements. That is, the implementation does not match the stated requirements. Bug fixes are another type of *corrective maintenance*.

The ability to evolve software over time is important—it enables us to use the software for long periods of time while still making improvements to it. *With the support of feature rich version control systems, along with software tracking (e.g., bug tracking) systems and iterative software development practices, continual change can be effectively managed, and even viewed as an asset that enables us to make discoveries and improvements that facilitate continuous improvement of the software.*

Most software is designed, built and maintained by teams of people. Team members may or may not be co-located and may or may not have the same work hours. Teams often have members with diverse backgrounds, skills, and interests. Some team members may have a technical background, similar to your own. Other team members may represent the client, or user, and may have limited or no software development knowledge. Furthermore, teams change over time. New developers join the team and existing team members move between projects or assume new responsibilities. The team-based nature of software development creates non-technical challenges that can greatly impact the success of the project. *Software engineering processes and tools support both technical and non-technical tasks, helping teams coordinate their work, communicate with each other, and control and track the many changes by many individuals necessary to build and maintain successful software systems.*

1.5 Essential Tools for Engineering Software

In this section we explore the types of software engineering tools you will soon use to organize and build various software artifacts. In general, each type of tool can be further organized based on its interface. The first type of interface is a *Command-Line Interface (CLI)*. The second type is a *Graphical User Interface (GUI)*. Users interact with a tool via the *CLI* by typing commands in a terminal or console window. Tools that provide a *GUI* interface use graphics, including windows, menus, buttons, scrollbars, etc. to enable users to interact with the tool via the keyboard, mouse, touchscreen, etc.

1.5.1 Organizing and Managing Software

Software Configuration Management (SCM) is an important part of software engineering. It refers to the management (i.e., tracking and controlling) of changes to the software. The underlying operating system on your computer provides a mechanism to organize software artifacts (and other documents). Most operating systems support a hierarchical *file system* that organizes files into folders, and creates hierarchies of nested folders. Each file contains the current version of the *artifact* or document. For example, if you create a spreadsheet on your computer, save it and exit the spreadsheet application, the filesystem contains the saved version of your spreadsheet. If you open the spreadsheet, make changes, and save and exit the application, the filesystem again contains the current version of your spreadsheet. The previous version no longer exists (unless you saved it to a different location or with a different file name).

An important component of software configuration management is version control. While maintaining a single version of each file saves disk space, it also makes it difficult to revert to a previous version of the file. Periodic and on-change backup systems such as those found in a word-processing app or provided by the underlying operating system help to mitigate this problem; however, these systems generally lack the more advanced features necessary to support software developed by teams of developers. For instance, if two people are working on the same file and their backups are stored on the same drive, changes made by one developer may overwrite changes by the other developer. And, depending on the number of backup versions stored, changes may be lost over time. If the developers' backups are stored on separate drives, then coordinating the changes from different backups becomes problematic. Similarly, filesystems do not include support for simultaneous edits to files. At best, the filesystem will warn the user that a file is already open for edits when it is opened for editing a second time, but it does not provide support for merging changes made simultaneously by different users.

Software engineers typically use a *Version Control System (VCS)* to manage changes to software artifacts. These systems typically provide support for merging changes to the same file and enable previous versions to be restored. They also provide support for versions of the artifact to evolve in parallel. Traditional version control systems followed a *centralized* model where a central *software repository* stores a copy of the files in a location that is accessible by all users (e.g., developers). Each user performs a *checkout* of the files, copying the files onto his or her computer. When the user has a change ready to share, a *commit* is performed to save the change on the server. Other users can then see and checkout the changed code to their own machines.



Figure 1.1: *Control-Flow Graph (CFG)* for the birthday cake algorithm.

In recent years, *Distributed Version Control Systems (DVCSs)* have been developed to enable software development without the need for a centralized repository. Instead of relying on a shared repository to store the project files, each user *clones* a copy of the project repository, including all of the history, to her own machine. While many projects continue to rely on a centralized repository to store the “official” version of the files, this practice is not required by a *DVCS*. When a user is ready to save changes, a *commit* is performed to save the changes to the local repository. When the user is ready to share his changes, the files are then *pushed* to a shared repository. The shared repository may be the centralized repository or another repository (e.g., another developer’s repository). Similarly when a user wants to integrate changes from other contributors, the changes are *pulled* from another repository into the user’s local repository.

Version control tools provide an interface to the version control system which enables users to maintain multiple versions of a file, share files with other users, and identify file differences across versions.

1.5.2 Reading and Writing Software Artifacts

At its most fundamental level, most source code is simply a sequence of characters. As such, any text editor can be used to read and write code. In fact, software developers used general text editors for many years as their primary tool for reading and writing source code. One of the primary downsides of using a generic text editor to write code is that it does not provide programming-language support features such as auto-completion or highlighting of programming constructs. It also does not support navigation between files based on the semantics, or meanings, of program elements. Sophisticated text editors such as word-processing software also attempt to “correct” source code to fit natural-language grammar, formatting, and punctuation which may not be consistent with the programming language conventions.

Given the limitations of general text editors, many software developers choose to use an *Integrated Development Environment (IDE)*. These tools compensate for the limitations of general text editors mentioned above, plus they are specifically designed to support software development by providing a single, integrated environment to eliminate the need to set up and integrate individual tools (e.g., version control, build tools,

debuggers, etc.). Many *IDEs* support multiple programming languages, and most modern *IDEs* provide a *GUI*.

Integrated development environment tools enable software developers to not only read and write code, but to perform many other tasks involving software.

1.5.3 Building Software

Although source code can be auto-generated, it is often written by humans, and as such is written to be read and updated by humans. Human-readable code, however, is not efficient for processing by an electronic device. In order to deploy code so that it can execute, or run, on a computer, it first needs to be built. The process of building, or compiling, code includes a sequence of steps that essentially translate the source code into a binary format that is executable. The build process also includes steps that analyze the program to ensure its correctness. As software systems have grown in size and complexity, build tools have grown in scope and sophistication, providing support for identifying and managing project dependencies, and building, testing, and deploying software systems.

Build tools enable software engineers to automatically translate source code to binary format. These tools also perform a variety of checks on the code and may deploy the software to the target environment.

1.5.4 Running Software

Software applications begin executing when we issue a command to the operating system telling it to launch the program. For example, to run an application on your phone or laptop, you tap or click on the application's icon. If the application is already running, and you click on its icon, the focus of the operating system will shift to the application. Similarly, to run a program with a *CLI*, you type a command that instructs the system to launch the application.

The underlying operating system is the 'tool' that enables software applications to run. We issue commands to the operating system to run software programs.

1.5.5 Analyzing Software

The last type of tool we discuss in this section is the set of tools used to analyze software programs. These tools enable software developers to check the correctness of the software, its performance, and other characteristics that can help determine if the application meets the specified requirements. Some analysis tools are run every time the application is built. For example, tools that determine if the source code follows the syntactic rules specified by the programming language, or tools that try to identify *regression errors*. Other analysis tools are run on-demand (e.g., when a new feature is added).

Analysis tools enable software developers to evaluate software for properties such as correctness, consistency and efficiency, to determine if the software meets its specified requirements.

1.6 Essential Skills & Knowledge for Engineering Software

Software engineers are problem solvers. They apply computational theories, models and techniques to the development of software in order to solve problems specified by their requirements. These activities involve problem formulation and analysis, solution design, and activities related to the implementation of the solution (i.e., developing code to organize, store and process the data necessary to solve the problem). Software engineers also solve engineering problems related to how the software is built, analyzed, stored,

documented, and maintained. As software systems have grown in size and complexity, these problems have become increasingly challenging. For instance, software engineering resources at organizations such as Google, Microsoft, and Facebook are specifically dedicated to finding ways to efficiently build, analyze and test the large bodies of evolving software that make up their products. To be an effective problem solver and software engineer requires a core set of technical skills and knowledge that include the ability to:

- Design a solution, given a computational problem.
- Read and understand code written by others.
- Know and apply basic algorithms and data structures.
- Analyze and evaluate alternative designs and solutions.
- Write correct *and maintainable* code.
- Design, write and use test cases.
- Use version control systems to store and share software artifacts.
- Learn and apply new models, technologies and techniques as they emerge in a rapidly evolving field.

Software engineers also need to have strong non-technical skills, such as the ability to work as part of a team and the ability to work with, and, in particular, communicate with non-software engineers—application developers, database analysts, and other technical specialists, as well as business managers and analysts, and clients who may have little understanding of how software is developed. For these, and other reasons, software engineers must also possess strong non-technical skills. These skills include the ability to:

- Work independently within a team.
- Actively listen to and ask questions of others.
- Communicate verbally and in writing with audiences with varying backgrounds and levels of technical expertise.
- Operate under tight deadlines
- Adapt to changes in the environment, software product, team composition, processes, etc.
- Ask for help and offer help to others.
- Persevere despite setbacks and failures.

While the lists included in this section are relatively short, and they contain somewhat generic skills, you should have an initial sense for what it takes to be a successful software engineer. It is interesting to note that this topic has recently become an area of research in the software engineering community. A relevant (and easy-to-read) paper that provides the insights of 59 experienced software engineers at Microsoft on this topic can be found in the paper, “What Makes a Great Software Engineer” by Paul Luo Li, Andrew J. Ko and Jiamin Zhu (available online).